

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3



Engineer to Excel



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3



Engineer to Excel



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3



Engineer to Excel



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3



Code of Conduct:

I Dhaanush Rajan (192421162) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWER

1) Banking Application – Testing as an Engineering Activity

Introduction

Testing should not be performed randomly. It must be treated as a systematic engineering activity involving planning, design, execution, measurement, and continuous improvement.

Redesigned Testing Approach

The banking team should follow these steps:

1. **Requirement Analysis** – Identify requirements such as login, fund transfer, balance enquiry, and transaction security.
2. **Test Planning** – Define scope, resources, tools, schedule, risks, and entry/exit criteria.
3. **Systematic Test Design** – Prepare test cases using:
 1. Equivalence Partitioning
 2. Boundary Value Analysis
 3. Decision Tables
 4. State Transition Testing
 5. Use-case testing
4. **Test Execution** – Execute test cases systematically and record actual results.
5. **Defect Reporting** – Record defects with steps, expected result, actual result, severity, and priority.
6. **Retesting and Regression Testing** – Verify fixes and ensure existing functions are not affected.
7. **Test Metrics** – Measure defect density, pass percentage, test coverage, and defect leakage.

Tools and Standards

Tools such as **Jira**, Selenium, Postman, JUnit, and CI/CD tools can be introduced. Standard test-case templates and defect-reporting formats should be followed.

Practical Example

For fund transfer:

Test	Expected Result
Transfer ₹1,000 with sufficient balance	Successful
Transfer ₹0	Rejected
Transfer greater than balance	Rejected
Invalid account number	Error message
Network failure during transfer	Transaction safely rolled back

Advantages

- Reduces missed defects.
- Provides repeatable testing.
- Improves test coverage.
- Makes testing measurable.
- Reduces production failures.

Conclusion

Replacing ad-hoc testing with **risk-based, systematic and model-based test design** makes banking software more reliable and secure.

2)ATM System – Structured Testing Process Across SDLC

A structured testing process for an ATM system should be followed throughout the Software Development Life Cycle (SDLC). It helps testers identify defects at an early stage and ensures that all ATM functions work correctly, securely, and consistently.

1. Requirement Phase:

The tester reviews requirements such as card insertion, PIN verification, balance enquiry, cash withdrawal, fund transfer, receipt generation, and card ejection. Any unclear or missing requirements are identified before development.

2. Design Phase:

Test scenarios and test cases are prepared according to the ATM design. Important conditions such as wrong PIN, insufficient balance, invalid amount, expired card, and network failure are considered.

3. Development Phase:

Developers perform unit testing for individual modules such as PIN validation and balance checking. Testers prepare detailed test cases for integration and system testing.

4. Integration and System Testing:

The interaction between the ATM machine, bank server, database, card reader, and cash dispenser is tested. The complete transaction flow is checked to ensure that data is transferred correctly between all components.

5. Acceptance Testing:

The final ATM system is tested against customer and business requirements. The system is approved only when important functions work correctly under normal and abnormal conditions.

ATM Withdrawal Example

For a ₹5,000 withdrawal, the following cases can be tested:

- Correct card + correct PIN + sufficient balance → Cash is dispensed
- Wrong PIN → Transaction is rejected
- Insufficient balance → Proper error message is displayed
- ATM has insufficient cash → Transaction is cancelled safely
- Invalid withdrawal amount → Amount is rejected
- Network failure → Transaction is safely handled without incorrect deduction
- Daily withdrawal limit exceeded → Transaction is rejected

Importance and Outcomes

A structured testing process provides consistent testing, better defect detection, improved security, and fewer transaction failures. It also ensures that defects found in one stage are corrected before moving to the next stage. Therefore, applying structured testing throughout the SDLC improves the reliability, accuracy, security, and overall quality of the ATM system.

3) Web Application – Errors, Defects and Failures

In a web application, error, defect, and failure are different stages of a problem. An error is a human mistake made during requirements, design, coding, or testing. A defect is the flaw introduced into the software because of that error. A failure occurs when the software executes the defect and produces an incorrect result visible to the user.

Definitions

- **Error:** A human mistake made during requirement analysis, design, coding, or testing.
- **Defect:** An incorrect or missing part of the software caused by an error.
- **Failure:** The incorrect behavior or output observed when the software is executed.

Example – Password Validation

Suppose the requirement states that the password must contain at least 8 characters. If the developer misunderstands the requirement and writes a condition allowing passwords with only 5 characters:

- Developer misunderstanding → **Error**
- Incorrect password validation code → **Defect**
- User successfully logging in with a 5-character password → **Failure**

Transaction Example

Suppose a banking web application should deduct ₹10,000, but only ₹1,000 is deducted.

- Incorrect coding or calculation → **Defect**
- ₹1,000 being deducted instead of ₹10,000 → **Failure**
- The user or developer making the mistake that caused the incorrect code → **Error**



Relationship

Human Error → Software Defect → Program Execution → Failure

This relationship helps testers understand the **root cause** of a problem instead of only identifying its visible effect.

Problems When Terms Are Confused

If testers use the word “error” for every problem:

- Developers may misunderstand the actual issue.
- Root-cause analysis becomes difficult.
- Defect reports and statistics may become inaccurate.
- Communication between testers and developers becomes unclear.
- Fixing and tracking defects becomes more difficult.

Important Defect Report Details

A proper defect report should contain:

1. Defect ID and module
2. Environment and preconditions
3. Steps to reproduce
4. Expected result
5. Actual result
6. Severity and priority
7. Screenshot, log, or supporting evidence
8. Reproducibility information

Conclusion

Clearly distinguishing error, defect, and failure helps the testing team communicate problems correctly and identify their root causes. It improves defect tracking, debugging, testing accuracy, and overall software quality.

4) Defect Clustering and Test Case Prioritization

Defect clustering is the testing principle that states that a small number of modules often contain a large number of defects. When most defects are found in a few modules, testers should concentrate more testing effort on those high-risk areas instead of testing every module equally.

1. Applying Testing Principles

The team should first analyze previous defect reports and identify modules with frequent failures. Testing should then follow a risk-based approach, giving higher priority to modules that are complex, frequently changed, business-critical, or have a history of defects.

Important principles include:

- Defect clustering – focus on modules where most defects occur.
- Risk-based testing – test high-risk functions first.
- Early testing – identify defects as early as possible.
- Exhaustive testing is impossible – prioritize important test conditions.
- Regression testing – retest affected modules after changes.

2. Test Planning Using Defect Clustering

Suppose an e-commerce application has modules such as:

- Login – 5 defects
- Product search – 8 defects
- Shopping cart – 30 defects
- Payment – 45 defects
- Profile – 4 defects

The payment and shopping cart modules have the highest defect concentration. Therefore, more testers, test cases, and execution time should be allocated to these modules.

3. Test Case Prioritization

Test cases can be prioritized based on:

1. Business importance – payment and checkout receive high priority.

2. Previous defect history – frequently failing modules are tested first.
3. Risk and impact – defects affecting money or security get high priority.
4. Complexity – complex modules require more testing.
5. Recent changes – modified modules need regression testing.
6. Customer usage – frequently used features should be tested early.

4. Improving Testing Efficiency

Defect clustering helps the team use limited testing time effectively. Instead of spending equal effort on all modules, testers concentrate on areas where defects are more likely to occur. This results in early defect detection, reduced testing time, better resource utilization, improved coverage of risky areas, and fewer production failures.

5. Real-Time Example

In an online banking application, historical testing may show that most defects occur in the fund transfer and payment modules. Testers can prioritize scenarios such as insufficient balance, duplicate transactions, incorrect account numbers, network failure, transaction limits, and concurrent transfers. Low-risk features such as changing the profile picture can be tested later.

Conclusion

Defect clustering and risk-based testing help the team identify where testing effort is most valuable. By prioritizing high-risk and frequently failing modules, the organization can detect important defects earlier, reduce delays, and improve overall testing efficiency and software quality.

5) Defect Repository and Defect Classification

A **defect repository** is a centralized place where information about software defects is stored and maintained. It contains details about defects found in the current project as well as defects from previous releases. This information helps testers identify repeated problems, plan regression testing, and improve future software quality.

1. Defect Origin

Defects can be introduced at different stages of the Software Development Life Cycle:

- **Requirements** – incorrect or incomplete requirements.

- **Design** – incorrect system or database design.
- **Coding** – programming or logic mistakes.
- **Integration** – problems between different modules.
- **Testing** – incorrect or missing test cases.
- **Deployment** – configuration or environment-related problems.

2. Defect Classification

Defects can be classified based on their nature:

- **Functional** – a feature does not work correctly.
- **Performance** – system response is slow.
- **Security** – unauthorized access or data protection problems.
- **Usability** – application is difficult to use.
- **Interface** – incorrect communication between modules or systems.
- **Database** – incorrect data storage or retrieval.
- **Compatibility** – problem occurs in a particular browser, OS, or device.
- **Logic** – incorrect calculation or business logic.
- **Configuration** – incorrect system or environment settings.

3. Example

Suppose an online banking application repeatedly fails during **payment processing because decimal values are not handled correctly**.

The defect repository may show that a similar payment defect occurred in an earlier release. Testers can use this historical information to create additional test cases for **decimal values, rounding, and different transaction amounts**.

4. Using Repository Data

Historical defect information helps the testing team to:

1. Create effective regression test cases.
2. Identify high-risk and frequently failing modules.
3. Improve requirements and design.
4. Prepare testing checklists.
5. Perform root-cause analysis.

6. Identify recurring defect patterns.
7. Prevent similar defects in future releases.

5. Improvements

A good defect repository should:

- Use a centralized repository.
- Assign a unique defect ID.
- Record the root cause of each defect.
- Store affected versions and modules.
- Link defects with related test cases.
- Record defect status such as Open, Fixed, Retest, and Closed.
- Analyze repeated defects and take preventive action.

Conclusion

A properly maintained defect repository and classification system helps the testing team learn from previous defects. It improves regression testing, supports root-cause analysis, identifies risky areas, and reduces the repetition of defects in future software releases.

6) Mobile Banking – Fund Transfer Crashes

A mobile banking application may crash during fund transfer because of problems in the application, network, API, or database. Since fund transfer is a critical banking function, it should be tested systematically under both normal and abnormal conditions. Testing should identify the cause of the crash and ensure that money is not lost or deducted incorrectly.

Possible Causes

The crash may occur due to:

- Invalid input such as incorrect account number.
- Insufficient memory or device resources.
- Network interruption during the transaction.
- API failure between the mobile app and banking server.
- Database errors while updating transaction details.
- Null or missing values in the application.

- Concurrent transactions occurring at the same time.
- Poor exception handling for unexpected conditions.

Systematic Testing Approach

The application should be tested at different levels:

1. Requirement Testing – Verify that fund-transfer requirements are clear and complete.
2. Unit Testing – Test individual transfer and validation functions.
3. API Testing – Check communication between the app and banking server.
4. Integration Testing – Verify interaction between app, API, database, and payment services.
5. System Testing – Test the complete transfer process.
6. Performance Testing – Check the system under heavy transaction loads.
7. Security Testing – Verify authentication, authorization, and transaction protection.
8. Regression Testing – Ensure that fixes do not introduce new problems.

Root-Cause Analysis

When a crash occurs, testers should collect crash logs, error messages, and stack traces and reproduce the problem under controlled conditions. The team should identify whether the root cause is in the application code, API, database, network, or device environment.

Improvements: The system can be improved by using:

- Automated regression testing
- API automation
- Crash and error monitoring
- Negative testing
- Network-condition testing
- Proper exception handling
- Database transaction validation
- Proper transaction rollback and status checking

Conclusion

Systematic testing helps identify both visible crashes and the underlying conditions that cause them. For a mobile banking application, it is especially important to ensure that every transaction is secure, accurate, recoverable, and not duplicated. This reduces production failures and improves customer trust and reliability.

7) Testing vs Debugging

Testing: Testing is the process of executing software to identify defects and evaluate whether it satisfies requirements.

Debugging: Debugging is the process of locating, analyzing, and correcting the cause of a defect.

Difference

Testing	Debugging
Finds defects	Finds causes
Mainly performed by testers	Mainly performed by developers
Compares actual and expected results	Examines code and logic
Produces defect reports	Produces code fixes

Example: A banking application deducts ₹2,000 instead of ₹1,000.

Testing: Identifies incorrect deduction.

Debugging: Developer examines the calculation and finds that the amount is multiplied twice.

If developers believe debugging replaces testing:

- Independent verification is reduced.
- Some defects remain undiscovered.
- Test coverage becomes poor.

Structured Approach

Test → Report → Analyze → Debug → Fix → Retest → Regression Test

Conclusion

Testing and debugging are complementary activities. Testing discovers problems, while debugging identifies and fixes their causes.

8)E-Commerce – Cross-Browser Testing

Problem: A web application may behave differently in Chrome, Edge, Firefox, Safari, or different mobile browsers. Causes

- Different browser rendering engines.
- JavaScript compatibility.
- CSS differences.
- Unsupported features.
- Different screen resolutions.
- Browser-specific APIs.

Testing Strategy

Test:

- Login
- Product search
- Shopping cart
- Checkout
- Payment
- Forms
- Images
- Responsive layouts

Tools

Suitable tools include:

- Selenium
- Playwright
- BrowserStack
- WebDriver
- Developer Tools

Techniques

- Compatibility testing

- Responsive testing
- Automated UI testing
- Exploratory testing
- Regression testing

Example: The checkout button works in Chrome but does not respond in Safari. Cross-browser testing identifies the issue before deployment.

Conclusion: A planned browser matrix combined with automation provides better coverage and improves application reliability.

9) Payroll System – Logical Defects

A payroll system may calculate an employee's salary incorrectly because of mistakes in business logic or calculation. This is mainly classified as a logical or functional defect. Since salary calculation is an important function, it must be tested carefully using different salary values and conditions.

1. Defect Classification

The problem is classified as a logical/functional defect because the program does not follow the correct salary calculation formula.

Correct formula:

$$\text{Net Salary} = \text{Basic Salary} + \text{Allowances} - \text{Deductions}$$

If the program incorrectly performs:

$$\text{Net Salary} = \text{Basic Salary} - \text{Allowances} - \text{Deductions}$$

the calculated salary will be wrong.

2. Testing Techniques

The following techniques can be used:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables

- Calculation Verification
- Data-Driven Testing
- Regression Testing

3. Example

For an employee:

- Basic Salary = ₹50,000
- Allowance = ₹5,000
- Deduction = ₹3,000

Expected:

$$₹50,000 + ₹5,000 - ₹3,000 = ₹52,000$$

If the system produces ₹42,000, it indicates a logical defect.

4. Prevention

- Perform unit testing for salary calculations.
- Conduct peer code reviews.
- Use automated regression tests.
- Maintain requirement traceability.
- Test boundary salary values.

Conclusion

Proper classification and testing of payroll defects help ensure accurate salary calculation and prevent financial errors from reaching production.

10) Random Testing → Structured Test Case Design

Random testing may miss important conditions because test cases are selected without a systematic plan. A better approach is to design test cases based on requirements, risks, and different input conditions. This provides better coverage and makes testing repeatable.

1. Structured Testing Process

The testing process should follow:

1. Analyze requirements.
2. Identify test conditions.
3. Define test scenarios.
4. Design test cases.
5. Define expected results.
6. Execute tests.
7. Record defects.
8. Perform regression testing.

2. Test Design Techniques

Useful techniques include:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Use-Case Testing
- Pairwise Testing
- Risk-Based Testing

3. **Example:** Suppose an ATM allows a maximum withdrawal of ₹20,000. Instead of randomly testing amounts, boundary testing can use:

- ₹19,999 → Valid
- ₹20,000 → Valid
- ₹20,001 → Invalid

This gives better coverage around the important boundary.

4. Advantages

Structured test design provides:

- Better test coverage
- Repeatability

- Traceability
- Reduced duplication
- Improved defect detection

Conclusion: Random testing should be replaced with requirements-based and risk-based test design so that important conditions are tested systematically.

11) Healthcare Application – Data Inconsistency

A healthcare application may contain different modules such as registration, appointment, billing, and medical records. If patient information is different between these modules, it can cause serious problems. Therefore, data consistency must be tested across the complete application.

1. Possible Causes

Data inconsistency may occur because of:

- Incorrect database mapping
- API synchronization errors
- Duplicate records
- Incorrect update logic
- Data-format mismatch
- Transaction failures

2. Testing Approach

Test the complete flow of patient information:

Registration → Appointment → Billing → Medical Record

For example, if a patient's name is changed during registration, the updated name should appear correctly in the appointment, billing, and medical-record modules.

3. Testing Techniques

- Integration Testing
- Database Testing
- API Testing
- Data Validation

- End-to-End Testing
- Regression Testing

4. Defect Reporting

A defect report should contain:

- Patient/test-data ID
- Module
- Steps to reproduce
- Expected data
- Actual data
- Database evidence
- Severity
- Environment

Conclusion: Systematic integration and data-integrity testing ensures that patient information remains accurate and consistent across all healthcare modules.

12)Repeated Defects Across Releases

When the same defects appear repeatedly in different software releases, it indicates that the development and testing team is not effectively using information from previous defects. A defect repository should be maintained to store historical defect information such as the defect description, root cause, affected module, release, severity, resolution, related test case, and preventive action. This information helps the testing team understand why defects occurred and take appropriate steps to prevent them from appearing again.

1. Defect Repository

A defect repository acts as a centralized source of information about previously identified defects. It should contain the defect description, root cause, affected module, release number, severity, resolution, related test case, and preventive action. By maintaining these details, testers can easily identify whether a newly discovered defect is similar to an earlier defect and can check how the previous problem was resolved.

2. Identifying Defect Patterns

The repository can be analyzed to identify modules where defects occur frequently. For example, suppose the payment module has the following defect history:

Release	Payment Defect
R1	12
R2	15
R3	13

This pattern shows that the payment module is a high-risk area because a large number of defects are repeatedly reported across releases. Therefore, the testing team should give more attention to payment-related functions during future releases.

3. Actions to Prevent Repeated Defects

The testing team should use historical information while preparing the test plan. Regression test suites should be created from previously fixed defects so that the same problems are checked in every future release. The team should also review the root cause of repeated defects and strengthen code reviews, testing checklists, and automated tests. Defect trend analysis can be performed regularly to identify modules that require additional testing.

4. Importance of Historical Defect Information

Historical defect information helps testers identify high-risk modules, frequently failing functions, and common causes of defects. It also helps in selecting test cases, improving regression testing, planning testing resources, and preventing similar problems. For example, if payment calculation defects have appeared in several releases, testers can create additional test cases for different payment amounts, invalid inputs, boundary values, and transaction conditions.

5. Real-Time Example

In an online banking application, suppose the payment module repeatedly produces transaction failures after new releases. By checking the defect repository, testers may find that similar failures occurred in earlier versions. The team can analyze the root cause, add the failed scenarios to the regression suite, automate those test cases, and perform additional testing whenever the payment module is modified.

Conclusion

A defect repository should not be used only as a place for storing defect records. It should become a defect-prevention tool that provides useful information for future test planning. By analyzing previous defects, creating regression tests, identifying risky modules, and performing root-cause analysis, the testing team can reduce repeated defects, improve testing efficiency, and increase the overall quality and reliability of the software.

13. Web-Based Learning System – Peak Usage Failure

A web-based learning system may work correctly under normal conditions but fail when thousands of students access it at the same time. This indicates that functional testing was performed, but sufficient **performance and load testing** was not carried out.

1. Testing Gap: The main problem is inadequate testing under high user load. The system should be tested under normal, peak, and extreme conditions.

2. Required Testing

- Load Testing
- Stress Testing
- Spike Testing
- Endurance Testing
- Scalability Testing

3. Example: If the system is expected to support **5,000 concurrent students**, testing can be performed with:

- 1,000 users
- 2,500 users
- 5,000 users
- 7,500 users

The team should measure:

- Response time
- CPU usage
- Memory usage
- Throughput
- Error rate

4. Coverage: Testing should include:

- Normal load
- Peak load
- Sudden traffic increase
- Long-duration usage
- Multiple simultaneous transactions

Conclusion

Functional correctness alone does not guarantee system reliability. Performance testing must be included to ensure that the learning system remains stable during **peak usage**.

14) Early Tester Involvement in SDLC

If testers are involved only after development is completed, many defects may be discovered late, making them more expensive and difficult to fix. Testing should follow the **shift-left approach**, where testers participate from the early stages of the SDLC.

1. Tester Activities

Requirements: Review requirements and identify unclear or missing conditions.

Design: Review architecture and identify integration risks.

Development: Prepare test cases and automation early.

Testing: Execute system and acceptance testing.

Deployment: Validate production readiness.

2. Practical Example

Suppose the requirement states:

"The user can transfer money."

A tester can ask:

- What is the maximum transfer amount?

- What happens if the network fails?
- What happens if the balance is insufficient?
- What happens if the user presses Transfer twice?

These questions can identify missing requirements before coding begins.

3. Benefits

- Early defect detection
- Lower correction cost
- Better requirements
- Improved collaboration
- Better test coverage

Conclusion: Testers should participate throughout the SDLC rather than being treated as only the final checkpoint. Early testing improves software quality and reduces development cost.

15) Incomplete Test Coverage

Incomplete test coverage occurs when a software system is tested only for some conditions while important scenarios are left untested. A system may work correctly for normal inputs but fail when users provide invalid, unexpected, or boundary values. Therefore, effective test planning should cover positive, negative, boundary, exceptional, and important real-world conditions. Proper test coverage helps testers identify defects before the software is released.

1. Causes of Incomplete Test Coverage

Incomplete coverage can occur for several reasons. Sometimes requirements are not clearly understood or some requirements are missed during test planning. Testers may also create only a small number of test cases because of limited time or resources. Lack of boundary testing, negative testing, and regression testing can also leave important defects undetected.

Common causes include:

- Missing or unclear requirements
- Insufficient number of test cases
- No boundary value testing
- Lack of negative testing

- Lack of regression testing
- Limited testing time and resources
- Failure to consider exceptional conditions

2. Example

Consider a login system. If the tester checks only a valid username and valid password, the basic functionality may appear to work correctly. However, many important conditions are not covered. The system should also be tested with empty fields, incorrect passwords, locked accounts, maximum-length inputs, and invalid or security-related inputs.

Important test cases include:

- Valid username and password
- Empty username
- Empty password
- Invalid username
- Invalid password
- Locked user account
- Maximum-length username or password
- Invalid or unexpected input
- Multiple unsuccessful login attempts

Testing these conditions provides better confidence that the login system behaves correctly in both normal and abnormal situations.

3. Techniques to Improve Test Coverage

Testers can use different test design techniques to identify missing scenarios. A Requirement Traceability Matrix (RTM) can be used to map each requirement to one or more test cases. Equivalence Partitioning divides input data into valid and invalid groups, while Boundary Value Analysis checks values near minimum and maximum limits.

Important techniques are:

1. Requirement Traceability Matrix (RTM)
2. Equivalence Partitioning
3. Boundary Value Analysis

4. Decision Table Testing
5. State Transition Testing
6. Negative Testing
7. Risk-Based Testing
8. Regression Testing

4. Coverage Measurement

Test coverage should be measured to determine how much of the system has been tested. Requirement coverage checks whether all requirements have corresponding test cases. Statement and branch coverage are commonly used in code-level testing, while functional coverage checks whether important system functions and scenarios have been tested.

Coverage can be measured using:

- Requirement coverage
- Statement coverage
- Branch coverage
- Condition coverage
- Functional coverage

5. Importance of Complete Test Coverage

Complete and systematic coverage helps identify defects that may otherwise reach production. It improves confidence in the software and ensures that both expected and unexpected user behaviors are considered. It also helps testers prioritize important areas and reduces the possibility of failures after deployment.

Conclusion

Incomplete test coverage can result in undetected defects and unexpected failures in real-world usage. By using proper test planning, requirement tracing, positive and negative testing, boundary analysis, and regression testing, testers can improve coverage and identify more defects. Adequate test coverage therefore improves **software reliability, quality, and defect detection**.

16) Logistics Application – Defect Reporting

Defect reporting is an important part of the software testing process. In a logistics application, defects may occur in areas such as delivery tracking, order processing, shipment status, and notifications. If defects are

reported with insufficient information, developers may not be able to reproduce or understand the problem easily. Therefore, a standardized defect-reporting process should be followed to provide clear and complete information.

1. Importance of a Good Defect Report

A good defect report gives developers enough information to reproduce the problem and identify its possible cause. It also helps the testing team track the defect from the time it is reported until it is fixed and verified. Clear reports reduce unnecessary communication between testers and developers and help the team resolve important defects faster.

2. Contents of a Good Defect Report

A defect report should contain all the important information related to the problem.

Important details include:

1. Defect ID
2. Clear defect title
3. Affected module
4. Test environment
5. Preconditions
6. Steps to reproduce
7. Expected result
8. Actual result
9. Severity
10. Priority
11. Screenshot, log, or supporting evidence
12. Test data
13. Defect status
14. Tester information
15. Build or release version

The steps to reproduce are especially important because they allow developers to perform the same actions and observe the problem.

3. Example of Defect Reporting

A poor defect report may simply state:

"Delivery tracking not working."

This report does not provide enough information for the developer to understand the exact problem.

A better defect report would state:

"Delivery status remains 'In Transit' after successful delivery confirmation."

The report should then include the required steps, test data, expected result, actual result, environment, and supporting screenshots or logs. This makes the defect easier to reproduce and investigate.

4. Severity and Priority

Severity indicates how seriously the defect affects the application. For example, a complete failure of delivery tracking may have high severity because an important function is unavailable. Priority indicates how quickly the defect should be fixed.

For example:

1. **High severity:** Delivery status cannot be updated for any shipment.
2. **Medium severity:** Status update is delayed under certain conditions.
3. **Low severity:** Minor spelling mistake in the delivery message.

Priority should be assigned according to business impact and release requirements.

5. Benefits of Standardized Defect Reporting

A standardized defect-reporting process provides several benefits to the development and testing team.

It provides:

1. Faster defect reproduction
2. Better communication between testers and developers
3. Easier defect tracking
4. Better prioritization of important defects
5. Reduced defect resolution time
6. Accurate testing and quality metrics

7. Better historical information for future testing
8. Easier regression testing

Conclusion

A clear and standardized defect report is essential for effective software testing. In a logistics application, detailed reporting helps developers quickly reproduce and fix problems related to delivery, shipment, and tracking. By including complete information such as steps, expected and actual results, severity, priority, logs, and test data, the team can **reduce communication gaps, improve defect resolution, and increase software quality.**

17) Process-Based Testing for Consistency

When different testers follow different testing methods, the quality of testing may become inconsistent. To solve this problem, the organization should follow a **standardized testing process** for all modules.

1. Standard Testing Process

The process can be:

Requirement Review → Test Planning → Test Design → Test Environment → Execution → Defect Reporting → Retesting → Regression → Closure

2. Standard Documents

The team should maintain:

- Test Plan
- Test Scenarios
- Test Cases
- Requirement Traceability Matrix
- Defect Reports
- Test Summary Report

3. Example

For every application module, testers should include:

- Positive test cases

- Negative test cases
- Boundary test cases
- Security test cases
- Regression test cases

4. Benefits

Standardization provides:

- Consistency
- Repeatability
- Traceability
- Better coverage
- Easier audits

Conclusion

Process-based testing ensures that testing quality does not depend on individual tester habits. A common process provides **consistent and repeatable testing across all modules**.

18) Untested Edge Cases – Boundary Testing

A system may fail after deployment because unusual or extreme input values were not tested. These conditions are called **edge cases**. Boundary Value Analysis is an effective technique for identifying defects around minimum and maximum limits.

1. Boundary Value Analysis

Suppose an application accepts an age between **18 and 60 years**.

Important test values are:

- 17 → Invalid
- 18 → Valid
- 19 → Valid
- 59 → Valid
- 60 → Valid
- 61 → Invalid

2. Why Edge Cases Matter

Normal inputs may work correctly, but defects may occur with:

- Minimum values
- Maximum values
- Empty values
- Zero
- Negative values
- Maximum-length strings
- Large transactions

3. Testing Approach

Testers should check:

- Minimum value
- Maximum value
- Just below the boundary
- Just above the boundary
- Invalid values

4. Benefits

Boundary testing helps identify defects that normal test cases may not detect. It is especially useful for applications involving **limits, amounts, ages, quantities, dates, and input lengths**.

Conclusion: Boundary Value Analysis provides a systematic way to test extreme and unusual inputs and helps prevent **edge-case failures after deployment**.

19. Defect Documentation and Repository

Defect documentation is an important part of software testing because it provides complete information about the defects identified during testing. Without proper documentation, defects can become difficult to track, reproduce, analyze, and resolve. A centralized **defect repository** should be maintained throughout the software development life cycle. It stores historical information about defects and helps testers and developers understand repeated problems, identify high-risk areas, and improve future testing.

1. Defect Repository

A defect repository is a centralized storage location where information about all identified defects is maintained. It allows the testing and development teams to track the complete life cycle of a defect, starting from its identification until its resolution and closure. Historical defect information can also be used for regression testing and future test planning.

2. Required Information

A defect repository should contain complete and accurate information about every defect.

These details help developers reproduce the problem and allow testers to verify whether the defect has been correctly resolved.

3. Defect Life Cycle

The repository can also be used to track the life cycle of a defect. A defect may initially be reported as **New** and then move through states such as Assigned, Open, Fixed, Retested, and Closed. If the problem still exists after the fix, the defect can be reopened.

This process ensures that every defect is properly monitored and no important issue is forgotten.

4. Benefits of a Defect Repository

A centralized repository provides several benefits to the testing team. It makes defect information available in one place and allows the team to analyze previous defects and identify common patterns.

The main benefits are:

1. **Defect tracking:** Helps monitor defects from reporting to closure.
2. **Trend analysis:** Identifies modules where defects occur frequently.
3. **Root-cause analysis:** Helps determine why defects repeatedly occur.
4. **Regression test creation:** Previous defects can be converted into regression test cases.
5. **Risk identification:** Frequently defective modules can be treated as high-risk areas.
6. **Release quality assessment:** Defect history helps evaluate the quality of a software release.
7. **Historical knowledge:** Previous defect information can support future testing activities.

5. Example

Suppose a software application has five releases and each release contains repeated defects in the login module. By analyzing the defect repository, testers can identify the login module as a **high-risk area**. The team can then create a dedicated login regression test suite containing valid login, invalid password, locked account, empty fields, password boundary, and multiple failed-attempt scenarios.

Whenever changes are made to the login module, these regression tests can be executed to check whether previous defects have been reintroduced.

6. Importance in Future Testing

Defect documentation should not be used only for recording and closing defects. The information should also be used to improve the testing process. Testers can study previous defects, identify frequently failing modules, update test cases, improve regression testing, and focus more testing effort on high-risk areas.

For example, if the same type of calculation defect occurs repeatedly, additional test cases and automated regression tests can be created for that functionality.

Conclusion

Proper defect documentation and a centralized defect repository help organizations convert individual defect reports into useful **testing knowledge**. By maintaining complete defect information, tracking defect history, analyzing patterns, performing root-cause analysis, and creating regression tests, teams can prevent similar defects from occurring again. This improves **software quality, testing efficiency, reliability, and release confidence**.

20) Banking System – Repeated Transaction Failures

Repeated banking transaction failures indicate weaknesses in the existing testing process and may affect customers, financial transactions, and the reliability of the banking system. Since banking applications handle money and sensitive information, a structured and risk-based testing strategy should be followed. First, all transaction requirements should be clearly identified, including transfer amount, account validation, balance checking, transaction limits, authentication, and failure handling. Requirement-based testing ensures that every important transaction condition is converted into suitable test cases.

Risk-based testing should then be used to give more attention to critical banking functions. Fund transfer, online payment, authentication, and account balance operations should receive higher testing priority because failures in these areas can directly affect customers and financial data. Testers should also use



techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing, and Negative Testing to identify both normal and abnormal transaction conditions.

Integration testing is also important because a banking transaction passes through multiple components. The complete flow, such as **Banking App → API → Transaction Server → Database → Notification Service**, should be tested to ensure that transaction information is correctly transferred and stored at every stage. For example, during a ₹10,000 transfer, a valid account with sufficient balance should result in a successful transaction, while insufficient balance or an invalid account should result in rejection. Network interruption should allow safe recovery, duplicate requests should not create duplicate transactions, daily limit violations should be rejected, and server timeouts should preserve the correct transaction state.

To reduce repeated failures, the organization should use automated API testing and automated regression testing so that important transaction scenarios are checked regularly. Continuous integration can execute tests whenever new changes are introduced, while a defect repository can store information about previously identified transaction failures. Risk-based prioritization and requirement traceability help ensure that critical requirements are continuously tested. Production monitoring can also identify transaction problems occurring in real-world usage and provide information for future testing.

Therefore, a structured and risk-based testing process can significantly reduce repeated banking transaction failures. By combining requirement-based testing, risk-based testing, integration testing, automation, regression testing, defect tracking, and production monitoring, the organization can improve **transaction reliability, security, defect detection, and customer confidence**, while reducing the possibility of previously fixed defects appearing again in future releases.